PAKP: Predictive Asymmetric KV-Paging via Centered Bounding-Box Lookahead for Memory-Constrained Transformer Inference

Akhil M
Independent Researcher
AI Lead, We Guide
India

***Abstract*—Long-context autoregressive Transformer inference is constrained by the linear growth of Key-Value (KV) cache memory with sequence length. On consumer-grade GPUs with limited VRAM, this growth forces either context truncation or complete KV-cache offloading to host memory, the latter incurring substantial PCIe transfer latency. This paper presents Predictive Asymmetric KV-Paging (PAKP), a memory-management architecture that exploits the asymmetric capacity of GPU VRAM and host RAM by maintaining historical KV representations in pinned host memory while retaining compact page-level descriptors on the GPU. PAKP introduces three integrated mechanisms: (1) a *centered bounding-box page descriptor* that stores per-page channel-wise minimum, maximum, and centered-mean Key representations, providing an upper-bound relevance score for each historical page; (2) a *cross-layer predictive lookahead router* that projects an approximate RoPE-aligned Query for a subsequent Transformer layer using the current layer's hidden state, enabling page selection before the target layer executes; and (3) *asynchronous DMA prefetching* via a dedicated CUDA stream to overlap host-to-device page transfers with GPU computation. A prototype implementation integrated into a pretrained Qwen2.5-1.5B-Instruct model on an NVIDIA RTX 4050 Laptop GPU (6 GB VRAM) demonstrates successful passkey retrieval from evicted historical pages, reducing the active KV-token count by approximately 79% relative to the full prompt length while operating within the GPU memory budget. Synthetic benchmarks indicate that the dual-stream prefetching mechanism hides a substantial fraction of PCIe transfer latency behind GPU computation. Limitations, including incomplete ablation results and single-test-case live evaluation, are discussed.**

***Index Terms*—KV-cache management, Transformer inference, memory offloading, attention sparsity, predictive prefetching, consumer GPU**

I. Introduction

Autoregressive Transformer language models [1] generate text one token at a time, reusing previously computed Key and Value representations through a mechanism known as the KV cache. During inference, each new token requires access to the Keys and Values of all preceding tokens to compute attention. The memory required for this cache grows linearly with the sequence length S, the number of layers L, the number of KV heads H_{kv}, and the head dimension d_h:

$$M_{\text{KV}} = 2 \cdot L \cdot H_{\text{kv}} \cdot d_h \cdot S \cdot B_{\text{elem}}, \tag{1}$$

where the factor of 2 accounts for both Key and Value tensors, and B_{elem} denotes the bytes per element (e.g., 2 for FP16).

On consumer-grade GPUs with limited VRAM—such as the NVIDIA RTX 4050 Laptop GPU with 6 GB—this linear scaling quickly exhausts available memory. A model with 28 layers, 8 KV heads, and a head dimension of 128 requires approximately 112 KiB per token in FP16 precision. At a context length of 32,768 tokens, the KV cache alone occupies approximately 3.50 GiB, which, combined with model weights and runtime overhead, exceeds the available GPU memory.

Existing approaches to this memory constraint fall into several categories. *Token eviction* methods such as StreamingLLM [2] and H_2O [3] permanently discard historical tokens based on attention-sink retention or cumulative attention scores, sacrificing access to the discarded context. *KV-cache compression* methods such as SnapKV [4] reduce the cache footprint by retaining only the most important KV positions per head. *Query-aware page selection* methods such as Quest [5] use min/max Key bounds to identify relevant KV-cache pages, reducing memory bandwidth requirements. *Full offloading* approaches such as FlexGen [6] move model weights and KV data to host memory or disk, accepting increased latency. *Serving-oriented paging* systems such as vLLM [7] use OS-inspired virtual memory management to reduce fragmentation in multi-request scenarios on high-end GPUs.

This paper presents **Predictive Asymmetric KV-Paging (PAKP)**, a memory-management architecture designed for single-request, long-context inference on consumer GPUs. PAKP operates at the intersection of KV-cache offloading and query-aware page retrieval, with the following integrated contributions:

1) A *centered bounding-box page descriptor* that stores per-page channel-wise minimum, maximum, and centered-mean Key representations in GPU memory, enabling upper-bound relevance scoring without transferring historical Key tensors.
2) A *cross-layer predictive lookahead router* that projects a RoPE-aligned approximate Query for a subsequent Transformer layer from the current layer's hidden state, allowing page-selection decisions to be made before the target layer begins execution.

3) An *asynchronous dual-stream prefetch pipeline* that overlaps host-to-device page transfers with GPU attention computation via a dedicated CUDA stream and pinned host memory.

The architecture maintains the complete historical KV cache in pinned host RAM while keeping only a compact descriptor index, attention sinks, and a local context window in GPU VRAM. Selected historical pages are transferred asynchronously based on the lookahead prediction. A prototype is implemented as a custom Hugging Face Transformers cache backend and evaluated on an NVIDIA RTX 4050 Laptop GPU with 6 GB VRAM and 32 GB system RAM.

The remainder of this paper is organized as follows. Section II reviews related work. Section III formalizes the memory-constrained inference problem. Section IV describes the PAKP architecture. Sections V and VI detail the descriptor design and lookahead mechanism. Section VII discusses implementation. Section VIII presents experimental results. Section IX analyzes results and Section X discusses limitations. Section XI concludes.

II. Background and Related Work

A. Transformer Attention and KV Caching

The multi-head self-attention mechanism [1] transforms an input representation $\mathbf{X} \in \mathbb{R}^{S \times d_{\text{model}}}$ into Query, Key, and Value representations through learned projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{Q} \in \mathbb{R}^{S \times d_k}, \tag{2}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{K} \in \mathbb{R}^{S \times d_k}, \tag{3}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad \mathbf{V} \in \mathbb{R}^{S \times d_v}, \tag{4}$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

The attention output is computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{5}$$

During autoregressive decoding, only the Query for the current token is computed; the Keys and Values of all preceding tokens are retrieved from the KV cache. This avoids redundant computation but requires storing all historical KV representations in accessible memory.

B. Grouped-Query Attention

Grouped-Query Attention (GQA) [8] reduces the number of Key and Value heads relative to Query heads by a factor G, where $G = H_Q/H_{\text{kv}}$. Each KV head serves G Query heads. This reduces the per-token KV memory by a factor of G compared to standard multi-head attention while maintaining the expressiveness of the full Query head count.

C. Rotary Position Embeddings

Rotary Position Embeddings (RoPE) [9] encode position information by rotating the Query and Key vectors as a function of their absolute position m:

$$\text{RoPE}(\mathbf{x}, m) = \mathbf{x} \odot \cos(m\theta) + \text{rotate}(\mathbf{x}) \odot \sin(m\theta), \tag{6}$$

where θ contains position-dependent frequency components. The resulting dot product between a Query at position m and a Key at position n depends on their relative distance $m - n$, providing implicit relative position encoding.

A consequence of RoPE is that stored Key vectors carry position-specific rotations. Any Query used for Key retrieval must be rotated with a consistent position to produce meaningful similarity scores.

D. KV-Cache Offloading

When GPU VRAM is insufficient for the complete KV cache, offloading to host memory (CPU RAM) is an alternative. FlexGen [6] demonstrated high-throughput LLM inference on a single GPU by offloading weights and KV data to CPU memory and disk. However, full-cache transfers across the PCIe interface introduce substantial latency. For a 3.50 GiB KV cache transferred at 11.81 GB/s (measured PCIe Gen4 bandwidth on the test hardware), a complete transfer requires approximately 296 ms per token—yielding a transfer-only ceiling of approximately 3.4 tokens per second.

E. Token Eviction and Attention Sinks

StreamingLLM [2] discovered that initial tokens in a sequence act as *attention sinks*, receiving disproportionately high attention weights regardless of their semantic content. By retaining these initial tokens alongside a sliding window of recent tokens, StreamingLLM enables unbounded-length streaming inference. However, all tokens outside the retained window are permanently discarded.

H_2O [3] identifies *heavy-hitter* tokens—those accumulating high cumulative attention scores—and retains them alongside recent tokens. While effective for maintaining generation quality, both approaches sacrifice access to evicted historical information.

F. Query-Aware KV-Cache Selection

SnapKV [4] observes that attention heads consistently focus on clustered important tokens identifiable from an observation window at the end of the prompt, enabling per-head KV-cache compression without fine-tuning.

Most relevant to the present work, Quest [5] groups KV-cache tokens into pages and tracks channel-wise minimum and maximum Key values per page. The incoming Query is used to estimate an upper bound on each page's attention contribution, and only the top-scoring pages are loaded. Quest operates within GPU HBM bandwidth optimization, whereas PAKP targets CPU-GPU offloading with predictive prefetching.

G. Paged KV-Cache Management

vLLM [7] introduced PagedAttention, which virtualizes KV-cache memory management using OS-inspired paging. KV blocks are allocated non-contiguously and mapped through block tables, reducing memory fragmentation. PagedAttention is designed for multi-request serving on datacenter GPUs and does not address CPU-GPU offloading or single-request consumer GPU constraints.

H. Positioning of PAKP

PAKP differs from the above approaches in several respects. Unlike StreamingLLM and H_2O, PAKP does not permanently discard historical tokens; they remain accessible in host memory. Unlike FlexGen, PAKP does not transfer the complete KV cache; it selectively retrieves relevant pages. Unlike Quest, PAKP operates across the CPU-GPU boundary and incorporates cross-layer predictive prefetching to hide transfer latency. Unlike vLLM, PAKP targets single-request inference on consumer hardware rather than multi-tenant serving.

The specific contribution of PAKP is the combination of centered bounding-box descriptors with cross-layer lookahead-driven prefetching in an asymmetric memory hierarchy. The bounding-box descriptor design shares conceptual similarities with Quest's min/max bounds, but PAKP additionally applies global Key centering to reduce the common-mode bias in Key representations, integrates the descriptor scoring with a predictive Query from a preceding layer, and schedules the resulting page transfers asynchronously to overlap with GPU computation.

III. Problem Formulation

A. Memory Constraints in Long-Context Inference

Consider an autoregressive language model with L Transformer decoder layers, hidden dimension d_{model}, H_Q Query attention heads, H_{kv} Key-Value heads, and head dimension $d_h = d_{\text{model}}/H_Q$. Let $G = H_Q/H_{\text{kv}}$ denote the Grouped-Query Attention (GQA) ratio.

During autoregressive generation at sequence position S, the model caches the Key and Value states for all preceding positions $1, \ldots, S$. In half-precision floating-point format (FP16/BF16, $B_{\text{elem}} = 2$ bytes), the memory occupied by the KV cache across all layers is:

$$M_{\text{KV}}(S) = 2 \cdot L \cdot H_{\text{kv}} \cdot d_h \cdot S \cdot B_{\text{elem}} \quad \text{[bytes]}. \tag{7}$$

For the specific 28-layer architecture evaluated in this work ($L = 28$, $H_{\text{kv}} = 8$, $d_h = 128$), the per-token KV memory requirement is:

$$\begin{aligned} M_{\text{token}} &= 2 \times 28 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 114{,}688 \text{ bytes} \approx 112.0 \text{ KiB/token}. \end{aligned} \tag{8}$$

Table I summarizes the memory requirements at representative context lengths.

TABLE I: KV-Cache Memory Scaling ($L = 28, H_{\text{kv}} = 8, d_h = 128$, FP16)

Context Length (S)	**Bytes**	**Size (GiB)**	**Size (GB)**
1 token	114,688	0.0001	0.0001
4,096 tokens	469,762,048	0.437	0.470
8,192 tokens	939,524,096	0.875	0.940
16,384 tokens	1,879,048,192	1.750	1.879
32,768 tokens	3,758,096,384	3.500	3.758
65,536 tokens	7,516,192,768	7.000	7.516

The total GPU memory footprint M_{total} during inference comprises static model parameters M_{weights}, runtime overhead M_{runtime} (CUDA context, PyTorch allocator reserved buffers, framework activations), and the dynamic KV cache M_{KV}:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{runtime}} + M_{\text{KV}}(S). \tag{9}$$

On an NVIDIA RTX 4050 Laptop GPU with 6.00 GB physical VRAM (5.71 GiB usable), a 4-bit or 16-bit model occupying 2.2 GB to 3.0 GB weights plus 0.5 GB to 1.0 GB runtime overhead leaves at most approximately 2.2 GB to 3.0 GB of free VRAM for the KV cache. Under this memory ceiling, an uncompressed KV cache exhausts VRAM ($M_{\text{total}} > V_{\text{GPU}}$) at approximately:

$$\begin{aligned} S_{\max} &= \frac{V_{\text{GPU}} - (M_{\text{weights}} + M_{\text{runtime}})}{M_{\text{token}}} \\ &\approx 20{,}000\text{–}28{,}000 \text{ tokens}. \end{aligned} \tag{10}$$

Context lengths beyond 32K tokens invariably trigger CUDA Out-Of-Memory (OOM) failures under full VRAM retention.

B. Host-to-Device Transfer Latency and PCIe Limits

An alternative to context truncation is offloading historical KV data to system RAM (32 GB capacity). However, moving KV data across the Peripheral Component Interconnect Express (PCIe) bus introduces substantial communication latency.

Let β_{PCIe} denote the effective host-to-device (H2D) transfer bandwidth. The transfer latency required to reload an offloaded KV cache of size M_{KV} per decoding step is:

$$T_{\text{transfer}} = \frac{M_{\text{KV}}}{\beta_{\text{PCIe}}}. \tag{11}$$

On the experimental system (Intel Core i7-13620H, PCIe Gen4), the measured H2D bandwidth using pinned host memory is:

$$\beta_{\text{PCIe}} = 11.81 \text{ GB/s}. \tag{12}$$

In contrast, internal GPU VRAM bandwidth is specified at:

$$\beta_{\text{VRAM}} = 192.0 \text{ GB/s}. \tag{13}$$

The internal VRAM bandwidth exceeds the PCIe transfer bandwidth by a factor of:

$$\frac{\beta_{\text{VRAM}}}{\beta_{\text{PCIe}}} = \frac{192.0}{11.81} \approx 16.26 \times . \tag{14}$$

For a 32,768-token context where $M_{\text{KV}} \approx 3.50$ GiB (3.76×10^9 bytes), naive full offloading would require:

$$T_{\text{transfer}}^{\text{full}} = \frac{3.50 \times 2^{30} \text{ bytes}}{11.81 \times 10^9 \text{ bytes/s}} \approx 0.318 \text{ s} = 318.2 \text{ ms/token}, \tag{15}$$

or approximately 296.36 ms under decimal gigabyte conversions. The theoretical throughput ceiling imposed solely by bus communication (excluding all GPU compute) is:

$$\text{TPS}_{\text{ceiling}} = \frac{1}{T_{\text{transfer}}^{\text{full}}} \approx 3.14\text{–}3.37 \text{ tokens/second}. \tag{16}$$

This transfer bottleneck renders full naive offloading impractical for interactive generation.

C. The Selective Retrieval Optimization Problem

Rather than transferring the complete sequence of S tokens, let the sequence be partitioned into fixed-size pages of B tokens each ($B = 64$). The total historical context consists of $P = \lfloor S/B \rfloor$ pages.

Autoregressive decoding exhibits dynamic attention sparsity [2], [5]: at any layer $l \in \{1, \ldots, L\}$, only a small subset of historical tokens contributes significant probability mass to the softmax distribution. Let $\mathcal{P} = \{0, 1, \ldots, P-1\}$ denote the set of historical pages. We seek a selection operator $\sigma_l : \mathbb{R}^{H_Q \times d_h} \to \mathcal{P}^K$ that selects $K \ll P$ pages such that:

$$\sum_{p \in \sigma_l(\mathbf{Q}_l)} \sum_{t \in p} \alpha_{l,t} \approx \sum_{t=1}^{S} \alpha_{l,t}, \tag{17}$$

where $\alpha_{l,t}$ denotes the exact attention weight assigned to token t by Query $\mathbf{Q}_l$.

If only K pages ($K \cdot B$ tokens) are transferred per layer, the data volume transferred per decoding step decreases by a factor of:

$$R_{\text{transfer}} = \frac{S}{K \cdot B}. \tag{18}$$

For $S = 32{,}768$, $B = 64$, and $K = 4$ (256 retrieved tokens), the data reduction ratio is:

$$R_{\text{transfer}} = \frac{32{,}768}{256} = 128\times. \tag{19}$$

The corresponding selective transfer time is reduced to:

$$T_{\text{transfer}}^{\text{selective}} \approx \frac{256 \times 114{,}688 \text{ bytes}}{11.81 \times 10^9 \text{ bytes/s}} \approx 2.49 \text{ ms/token}. \tag{20}$$

Because typical single-token GPU forward compute per layer ranges between 0.7 ms and 1.5 ms (aggregating to 20 ms–25 ms across all layers), selective transfer latency satisfies:

$$T_{\text{transfer}}^{\text{selective}} \ll T_{\text{compute}}^{\text{total}}. \tag{21}$$

The core challenge then shifts to two fundamental requirements:

1) **Identification without full Key transfer:** Determining which K pages to retrieve must be performed using GPU-resident data structures whose memory footprint is negligible compared to the full KV cache.
2) **Hiding transfer latency:** If page selection is performed synchronously at layer l using $\mathbf{Q}_l$, the GPU must stall while waiting for the selected pages to arrive. Hiding this latency requires *speculatively predicting* the requirements of layer $l+1$ during the execution of layer l.

This motivates the PAKP architecture described next.

IV. PAKP Architecture

A. Architectural Overview

Predictive Asymmetric KV-Paging (PAKP) organizes memory across a two-tier hierarchy to balance capacity and communication efficiency. Figure 1 illustrates the system architecture.

The architecture defines two distinct memory tiers:

- **Tier 1 — GPU VRAM (6.0 GB):** Acts as a fast, capacity-bounded working memory. It stores static model weights, CUDA runtime allocations, attention sinks ($W_{\text{sink}} = 4$ tokens), a sliding local context window ($W_{\text{local}} = 256$ to 512 tokens), compact page descriptors (≈ 28 MB), and small prefetch staging buffers (≈ 1.1 MB).
- **Tier 2 — Pinned Host RAM (32.0 GB):** Acts as an expansive historical repository allocated via page-locked host memory (`cudaMallocHost` or PyTorch `pin_memory=True`). It stores the complete collection of evicted Key and Value pages across all layers ($P_{\max} = 512$ pages, corresponding to 32,768 tokens in 3.50 GiB).

B. Three-Component Working Context

Rather than computing attention over the unmanageable full sequence of length S, the GPU computes self-attention exclusively over a dynamically assembled active context of size $S_{\text{active}} \ll S$:

$$\mathbf{K}_{\text{active}}^{(l)} = \mathbf{K}_{\text{sink}}^{(l)} \oplus \mathbf{K}_{\text{prefetch}}^{(l)} \oplus \mathbf{K}_{\text{local}}^{(l)}, \tag{22}$$

$$\mathbf{V}_{\text{active}}^{(l)} = \mathbf{V}_{\text{sink}}^{(l)} \oplus \mathbf{V}_{\text{prefetch}}^{(l)} \oplus \mathbf{V}_{\text{local}}^{(l)}, \tag{23}$$

where $\oplus$ denotes concatenation along the sequence dimension. The components are:

1) **Attention Sinks ($\mathbf{K}_{\text{sink}}, \mathbf{V}_{\text{sink}}$):** The initial $W_{\text{sink}} = 4$ tokens. Following [2], initial tokens serve as mathematical sinks that absorb unallocated attention mass; retaining them permanently in VRAM prevents softmax entropy collapse.
2) **Local Context Window ($\mathbf{K}_{\text{local}}, \mathbf{V}_{\text{local}}$):** The most recent W_{local} tokens (e.g., 256 or 512 tokens). These tokens capture immediate syntactic structure, recency bias, and local co-reference.
3) **Prefetched Historical Pages ($\mathbf{K}_{\text{prefetch}}, \mathbf{V}_{\text{prefetch}}$):** Exactly $K = 4$ historical pages ($4 \times 64 = 256$ tokens) identified by the lookahead router as carrying high relevance to the incoming Query.

The active sequence length in VRAM is constant during steady-state decoding:

$$S_{\text{active}} = W_{\text{sink}} + K \cdot B + W_{\text{local}}. \tag{24}$$

For $W_{\text{sink}} = 4$, $K = 4$, $B = 64$, and $W_{\text{local}} = 256$, $S_{\text{active}} = 4 + 256 + 256 = 516$ tokens (or 552 tokens in configurations with padding/boundary tokens). This bounds the decoding KV-cache footprint in VRAM to under 60 MB, independent of whether the total context length S is 4K, 32K, or 64K tokens.

C. Historical KV Eviction Pipeline

The transition from full-context prefill to paged autoregressive decoding occurs via a one-time eviction routine depicted in Figure 2:

1) **Full Prefill Accumulation:** During prefill, all prompt tokens are processed through the Transformer backbone to allow full attention visibility across the prompt.

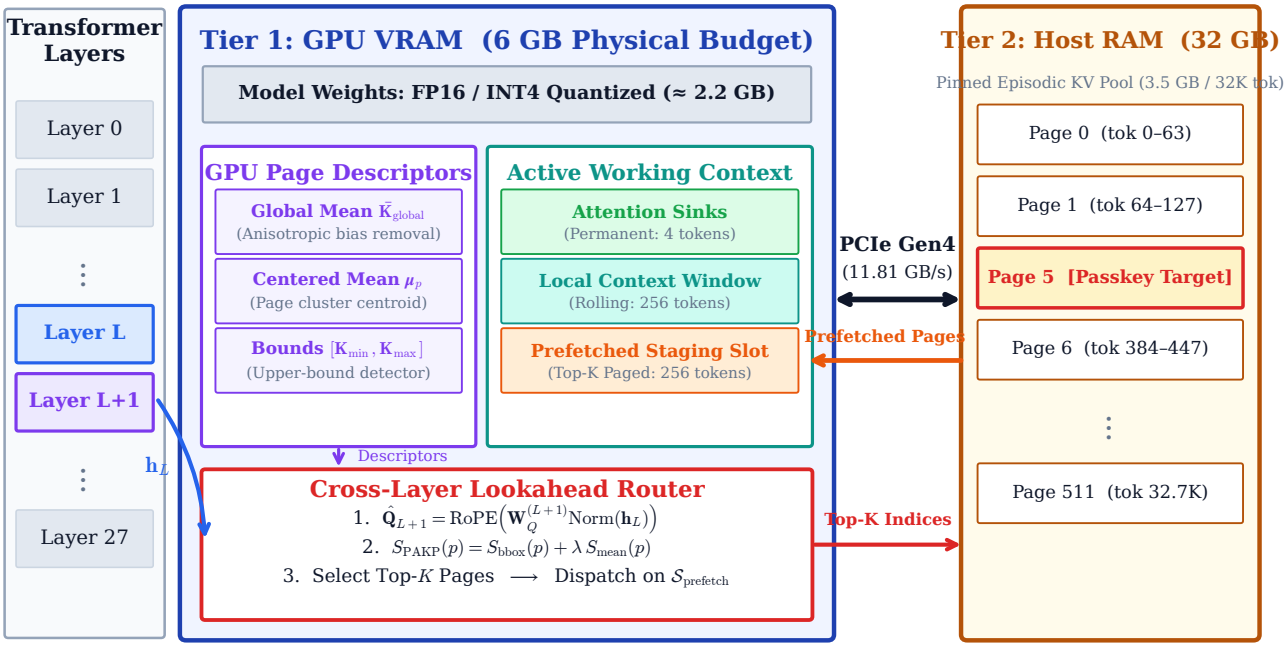


Fig. 1: PAKP system architecture. Tier 1 (GPU VRAM) retains static model parameters, attention sinks, a rolling local window, compact GPU page descriptors, and staging buffers. Tier 2 (Pinned Host RAM) holds the complete episodic historical KV pool. The lookahead router at Layer L predicts the Query for Layer $L+1$ and triggers asynchronous DMA prefetching over a dedicated CUDA stream.

2) **Context Partitioning:** Let S_{prompt} be the prompt length. The cache is segmented into:

$$\text{Sinks:} \quad [0, W_{\text{sink}}), \tag{25}$$

$$\text{Middle History:} \quad [W_{\text{sink}}, W_{\text{sink}} + P \cdot B), \tag{26}$$

$$\text{Local Window:} \quad [W_{\text{sink}} + P \cdot B, S_{\text{prompt}}), \tag{27}$$

where $P = \lfloor (S_{\text{prompt}} - W_{\text{sink}} - W_{\text{local}})/B \rfloor$.

3) **Descriptor Computation:** For each layer, the global Key mean across all historical tokens is computed. Centered mean, minimum, and maximum descriptors are generated for each page $p \in \{0, \ldots, P-1\}$ and retained in VRAM.

4) **Host Eviction:** Raw FP16 Key and Value tensors for all P pages are transferred to pinned host memory via non-blocking DMA. The corresponding VRAM allocations are freed via `torch.cuda.empty_cache()`.

D. Asynchronous Dual-Stream Coordination

To prevent host-to-device transfers from blocking the GPU compute pipeline, PAKP employs dual CUDA streams:

- **Compute Stream** ($\mathcal{S}_{\text{compute}}$)**:** The primary default CUDA stream executing forward layer compute (input layernorm, multi-head attention over active KV, and feed-forward SwiGLU MLP).
- **Prefetch Stream** ($\mathcal{S}_{\text{prefetch}}$)**:** An independent background CUDA stream dedicated to executing non-blocking DMA transfers of selected page buffers:

$$\mathbf{K}_{\text{host}}^{(l)}[:, \text{top_idx}] \xrightarrow{\text{PCIe DMA}} \mathbf{K}_{\text{staging}}^{(l)}.$$

Synchronization between the streams is managed via per-layer CUDA events E_l. Figure 3 illustrates the overlapping timeline:

1) During execution of Layer L, the lookahead hook predicts Query $\hat{\mathbf{Q}}_{L+1}$ and selects K target pages.
2) The transfer of these K pages is enqueued on $\mathcal{S}_{\text{prefetch}}$, followed immediately by recording event E_{L+1}.
3) $\mathcal{S}_{\text{compute}}$ continues computing Layer L's attention and MLP without interruption.
4) Prior to entering Layer $L+1$'s attention kernel, $\mathcal{S}_{\text{compute}}$ issues a stream wait on E_{L+1}:

$$\mathcal{S}_{\text{compute}}.\text{wait_event}(E_{L+1}). \tag{28}$$

5) If the DMA transfer completes while Layer L is executing, $\mathcal{S}_{\text{compute}}$ proceeds into Layer $L+1$ with zero pipeline stalls.

V. Centered Bounding-Box Page Descriptors

A. Motivation and Anisotropic Bias

A fundamental challenge in block-level KV retrieval is summarizing $B = 64$ Key vectors into a compact descriptor

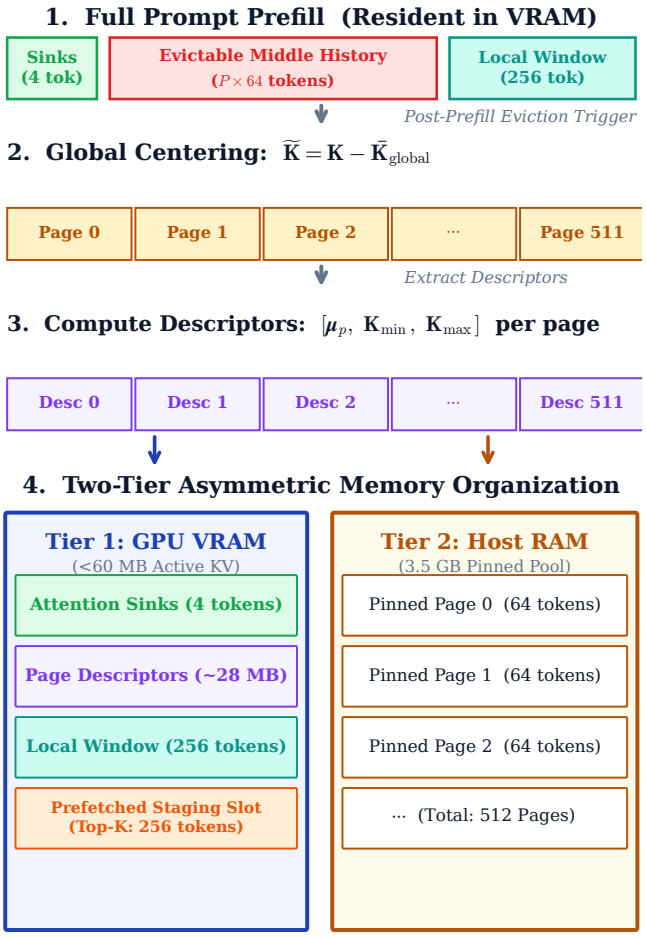


Fig. 2: Historical KV eviction pipeline. Following prompt prefill, the prompt context is partitioned into sinks, evictable middle history, and local window. Historical tokens are partitioned into 64-token pages, centered relative to the global Key mean, and summarized into compact descriptors. Raw KV pages are transferred to pinned host RAM.

that reliably indicates whether any individual token in the page has high relevance to an incoming Query.

Prior work in information retrieval and early KV-cache experiments commonly employ the mean centroid:

$$\boldsymbol{\mu}_p^{\text{naive}} = \frac{1}{B} \sum_{i=1}^{B} \mathbf{K}_{p,i}. \tag{29}$$

However, language model Key representations exhibit pronounced anisotropic bias: representations across tokens share large common-mode components along specific channels (e.g., embedding drift and positional artifacts). When evaluating inner products with $\boldsymbol{\mu}_p^{\text{naive}}$, the common directional bias dominates the score. Furthermore, in sparse retrieval tasks where a single high-salience "needle" token is embedded within 63 background tokens, mean pooling dilutes the needle signal by a

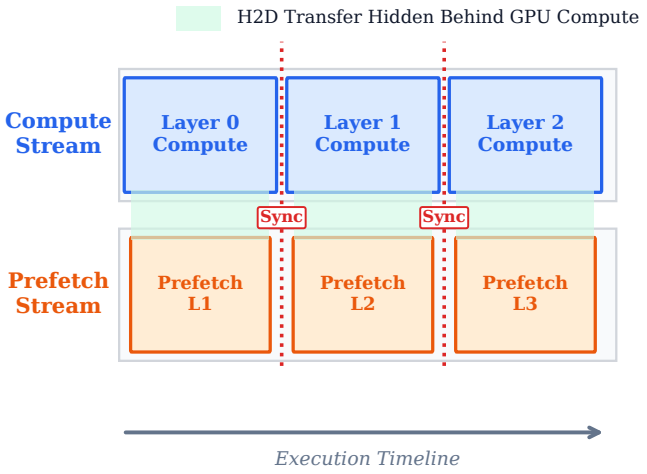


Fig. 3: Dual-stream asynchronous execution timeline. While the Compute Stream executes Layer L, the Prefetch Stream asynchronously transfers the predicted Top-K pages for Layer $L+1$ across the PCIe bus. CUDA events synchronize streams before Layer $L+1$ attention begins.

factor of $1/B = 1/64 \approx 0.0156$, causing mean-based routing to miss critical historical pages.

To preserve both average semantic orientation and isolated high-magnitude activations, PAKP introduces a *centered bounding-box descriptor* with three complementary components.

B. Mathematical Formulation

Let layer l contain P historical pages, each comprising B Key tokens $\mathbf{K}_{p,i} \in \mathbb{R}^{H_{\text{kv}} \times d_h}$ for $p \in \{0, \ldots, P-1\}$ and $i \in \{1, \ldots, B\}$.

1) Global Mean Centering: First, a layer-wide global Key mean is computed across all historical tokens:

$$\bar{\mathbf{K}}_{\text{global}}^{(l)} = \frac{1}{P \cdot B} \sum_{p=0}^{P-1} \sum_{i=1}^{B} \mathbf{K}_{p,i}^{(l)} \quad \in \mathbb{R}^{H_{\text{kv}} \times 1 \times d_h}. \tag{30}$$

Each Key vector is transformed into centered representation space:

$$\widetilde{\mathbf{K}}_{p,i}^{(l)} = \mathbf{K}_{p,i}^{(l)} - \bar{\mathbf{K}}_{\text{global}}^{(l)}. \tag{31}$$

Centering eliminates the shared directional bias across channels, ensuring that descriptor inner products reflect distinctive token features rather than background norms.

2) Three-Component Page Descriptors: For each page p, three descriptors are computed across the centered tokens $\widetilde{\mathbf{K}}_{p,i}$:

1) **Centered Mean Centroid ($\boldsymbol{\mu}_p$):**

$$\boldsymbol{\mu}_p^{(l)} = \frac{1}{B} \sum_{i=1}^{B} \widetilde{\mathbf{K}}_{p,i}^{(l)} \quad \in \mathbb{R}^{H_{\text{kv}} \times d_h}. \tag{32}$$

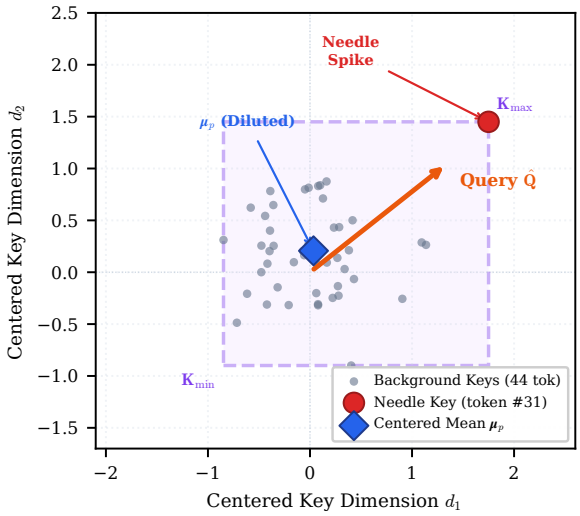


Fig. 4: Two-dimensional geometric illustration of the centered bounding-box descriptor. Centering removes common-mode bias. The bounding box $[\mathbf{K}_{\min}, \mathbf{K}_{\max}]$ captures channel-wise extrema contributed by isolated needle tokens that are diluted in the mean centroid $\boldsymbol{\mu}_p$. For any query $\hat{\mathbf{Q}}$, the bounding-box score provides an upper bound on token inner products.

2) **Channel-wise Minimum ($\mathbf{K}^{(p)}_{\min}$):**

$$\mathbf{K}^{(p,l)}_{\min,d} = \min_{1\le i\le B} \widetilde{\mathbf{K}}^{(l)}_{p,i,d} \quad \in \mathbb{R}^{H_{\text{kv}}\times d_h}. \tag{33}$$

3) **Channel-wise Maximum ($\mathbf{K}^{(p)}_{\max}$):**

$$\mathbf{K}^{(p,l)}_{\max,d} = \max_{1\le i\le B} \widetilde{\mathbf{K}}^{(l)}_{p,i,d} \quad \in \mathbb{R}^{H_{\text{kv}}\times d_h}. \tag{34}$$

These descriptors are stored permanently in GPU VRAM across all L layers.

C. Bounding-Box Scoring and Upper-Bound Property

Given an incoming Query $\mathbf{Q} \in \mathbb{R}^{H_Q\times d_h}$, let $\mathbf{Q}$ be grouped according to the GQA head mapping: $\mathbf{Q} \in \mathbb{R}^{H_{\text{kv}}\times G\times d_h}$, where $G = H_Q/H_{\text{kv}}$ is the group size.

For each KV head $h \in \{1,\dots,H_{\text{kv}}\}$, query head $g \in \{1,\dots,G\}$, and feature dimension $d \in \{1,\dots,d_h\}$, the potential inner-product contribution of any token inside page p's bounding box satisfies:

$$Q_{h,g,d}\widetilde{K}_{p,i,h,d} \le \max\left(Q_{h,g,d}K^{(p)}_{\max,h,d},\, Q_{h,g,d}K^{(p)}_{\min,h,d}\right). \tag{35}$$

Summing across all feature dimensions $d \in \{1,\dots,d_h\}$ yields the *bounding-box upper-bound score*:

$$S_{\text{bbox}}(p,h,g) = \sum_{d=1}^{d_h} \max\left(Q_{h,g,d}K^{(p)}_{\max,h,d},\, Q_{h,g,d}K^{(p)}_{\min,h,d}\right). \tag{36}$$

Proposition 1 (Upper Bound on Centered Dot Product). *For any centered Key vector $\widetilde{\mathbf{K}}_{p,i} \in \mathbb{R}^{d_h}$ in page p, the centered dot product with query head $\mathbf{Q}_{h,g}$ satisfies:*

$$\mathbf{Q}_{h,g}\cdot\widetilde{\mathbf{K}}_{p,i,h} \le S_{\text{bbox}}(p,h,g). \tag{37}$$

Proof: By definition of the channel-wise extrema, $K^{(p)}_{\min,h,d} \le \widetilde{K}_{p,i,h,d} \le K^{(p)}_{\max,h,d}$ for all $i \in \{1,\dots,B\}$ and all $d \in \{1,\dots,d_h\}$. If $Q_{h,g,d} \ge 0$, then $Q_{h,g,d}\widetilde{K}_{p,i,h,d} \le Q_{h,g,d}K^{(p)}_{\max,h,d}$. If $Q_{h,g,d} < 0$, then $Q_{h,g,d}\widetilde{K}_{p,i,h,d} \le Q_{h,g,d}K^{(p)}_{\min,h,d}$. In either case:

$$Q_{h,g,d}\widetilde{K}_{p,i,h,d} \le \max\left(Q_{h,g,d}K^{(p)}_{\max,h,d},\, Q_{h,g,d}K^{(p)}_{\min,h,d}\right).$$

Summing this inequality over $d = 1,\dots,d_h$ establishes the proposition. ∎

Remark on Exactness versus Bounding. We emphasize a key distinction: S_{bbox} is an *upper bound* on the maximum possible dot product achievable within the hyper-rectangle spanned by $[\mathbf{K}_{\min}, \mathbf{K}_{\max}]$. It does not generally equal the exact dot product of any individual physical token, because the channel-wise extrema across different dimensions d may originate from different tokens within the page. However, if a page contains a high-salience needle token whose coordinates push the bounding box in the direction of $\mathbf{Q}$, S_{bbox} will be large.

Centering and Ranking Invariance. The uncentered dot product between $\mathbf{Q}$ and the original Key $\mathbf{K}$ is:

$$\mathbf{Q}\cdot\mathbf{K}_{p,i} = \mathbf{Q}\cdot(\widetilde{\mathbf{K}}_{p,i}+\bar{\mathbf{K}}_{\text{global}}) = \mathbf{Q}\cdot\widetilde{\mathbf{K}}_{p,i}+\mathbf{Q}\cdot\bar{\mathbf{K}}_{\text{global}}. \tag{38}$$

Because the term $\mathbf{Q}\cdot\bar{\mathbf{K}}_{\text{global}}$ is identical across all historical pages $p \in \{0,\dots,P-1\}$, ranking pages according to centered upper bounds produces an identical relative ordering to ranking them according to uncentered bounds shifted by the global mean dot product.

D. Composite Scoring and Page Selection

To combine the peak-detection capability of the bounding box with the overall semantic theme captured by the mean centroid, PAKP computes a composite page score:

$$S_{\text{PAKP}}(p) = \max_{h,g} S_{\text{bbox}}(p,h,g) + \frac{\lambda}{H_{\text{kv}}G}\sum_{h=1}^{H_{\text{kv}}}\sum_{g=1}^{G}\mathbf{Q}_{h,g}\cdot\boldsymbol{\mu}_{p,h}, \tag{39}$$

where λ controls the weight of the centered mean component (empirically set to $\lambda = 0.50$ in the live implementation).

The router then selects the Top-K pages with the highest composite scores:

$$\mathcal{P}_{\text{selected}} = \text{Top-K}\left(\{S_{\text{PAKP}}(p)\}_{p=0}^{P-1},\, K\right). \tag{40}$$

E. Memory Footprint of Page Descriptors

Storing descriptors in VRAM avoids transferring Key data during search. For L layers, P pages, H_{kv} heads, and head dimension d_h, the storage required for all three descriptors $(\boldsymbol{\mu}, \mathbf{K}_{\min}, \mathbf{K}_{\max})$ in FP16 is:

$$M_{\text{desc}} = 3 \times L \times P \times H_{\text{kv}} \times d_h \times 2 \text{ bytes}. \tag{41}$$

For $L = 28$, $H_{\text{kv}} = 8$, $d_h = 128$, and $P = 512$ pages (32,768 tokens):

$$M_{\text{desc}} = 3 \times 28 \times 512 \times 8 \times 128 \times 2 \approx 88.08 \text{ MiB}. \tag{42}$$

In a mean-only indexing configuration, $M_{\text{desc}} \approx 29.36\,\text{MiB}$. Compared to the $3.50\,\text{GiB}$ raw KV cache, the descriptor footprint occupies under 2.5% of the original KV memory, making it practical to retain entirely in GPU VRAM.

VI. Cross-Layer Predictive Lookahead

A. Motivation and the Lookahead Gap

While the centered bounding-box descriptor enables fast page scoring, synchronous evaluation at Layer L using the true Query $\mathbf{Q}_L$ would force the GPU to stall until the requested pages arrive over PCIe. To achieve zero-stall execution, the required pages for Layer $L{+}1$ must be identified and requested while Layer L is executing.

This necessitates predicting the retrieval requirements of Layer $L+1$ in advance (Figure 5). In standard Transformer architectures, Layer L computes:

$$\mathbf{a}_L = \text{Attention}\left(\mathbf{Q}_L, \mathbf{K}_L, \mathbf{V}_L\right), \tag{43}$$

$$\mathbf{h}_L^{\text{mid}} = \mathbf{h}_L + \mathbf{a}_L \mathbf{W}_O^{(L)}, \tag{44}$$

$$\mathbf{h}_{L+1} = \mathbf{h}_L^{\text{mid}} + \text{MLP}_L\left(\mathbf{h}_L^{\text{mid}}\right), \tag{45}$$

where $\mathbf{h}_L$ is the input hidden state to Layer L, and $\mathbf{h}_{L+1}$ is the input hidden state to Layer $L+1$.

The true Query at Layer $L+1$ is defined by:

$$\mathbf{Q}_{L+1} = \text{RoPE}\left(\mathbf{W}_Q^{(L+1)}\,\text{Norm}_{L+1}\left(\mathbf{h}_{L+1}\right), p\right). \tag{46}$$

However, at the beginning of Layer L, $\mathbf{h}_{L+1}$ has not yet been computed because the attention and MLP sublayers of Layer L have not executed.

B. Residual Stream Continuity and Query Estimation

Transformer residual architectures exhibit strong directional continuity across adjacent layers. Because each layer computes a perturbation $\Delta\mathbf{h}_L = \mathbf{h}_{L+1} - \mathbf{h}_L$ whose norm is typically small relative to the residual stream norm $\|\mathbf{h}_L\|$, the input state $\mathbf{h}_L$ serves as a first-order approximation of $\mathbf{h}_{L+1}$:

$$\mathbf{h}_{L+1} \approx \mathbf{h}_L. \tag{47}$$

PAKP constructs the *predictive lookahead Query* $\hat{\mathbf{Q}}_{L+1}$ directly from the incoming hidden state $\mathbf{h}_L$ by applying Layer $L+1$'s input normalization and Query projection weights:

$$\hat{\mathbf{Q}}_{L+1} = \mathbf{W}_Q^{(L+1)}\,\text{Norm}_{L+1}\left(\mathbf{h}_L\right). \tag{48}$$

Critical Implementation Detail: Layer Indexing in Pre-Hooks. In the PyTorch implementation, hooks are registered as `register_forward_pre_hook` on each layer module `model.model.layers[l_idx]`. When the pre-hook executes at Layer l_idx, the incoming argument is the hidden state entering Layer l_idx. For $l_idx \geq 1$, this state is precisely the output of Layer $l_idx - 1$.

The lookahead hook uses this state to project into the target layer $t_idx = l_idx + 1$. Thus, the practical approximation computes:

$$\hat{\mathbf{Q}}_{L+1} = \mathbf{W}_Q^{(L+1)}\,\text{Norm}_{L+1}\left(\mathbf{h}_{L-1}^{\text{out}}\right), \tag{49}$$

which is one layer prior to the ideal post-Layer-L state. The empirical validity of this approximation is evaluated in Section VIII-B2.

Layer 0 Initialization. At Layer 0, the hook receives the initial token embedding $\mathbf{h}_0$. Because no previous layer has executed, Layer 0 cannot rely on a prior lookahead. To initialize the pipeline, the pre-hook at Layer 0 targets both Layer 0 and Layer 1:

$$\text{target_layers} = [0, 1] \quad \text{for } l_idx = 0. \tag{50}$$

For Layer 0, the projection uses $\mathbf{h}_0$, which is the exact input state. For Layer 1, the projection uses $\mathbf{h}_0$ as an approximation of $\mathbf{h}_1$. For all subsequent layers $l_idx \geq 1$, the target is $t_idx = l_idx + 1$.

C. RoPE Alignment

In modern architectures such as Qwen2.5 and Llama-3, Query and Key vectors undergo Rotary Position Embedding (RoPE) before attention is computed. As established in Section II-A, the stored Key vectors in the historical page pool carry position-specific rotary transformations:

$$\mathbf{K}_{p,i} = \text{RoPE}\left(\mathbf{X}_{p,i}\mathbf{W}_K,\, m_{p,i}\right), \tag{51}$$

where $m_{p,i}$ is the absolute position of token i in page p.

An early failure mode identified during the development of PAKP (documented in Section VIII-C1) arose from comparing an unrotated lookahead Query against position-rotated Key descriptors. Because RoPE applies non-trivial frequency-dependent coordinate rotations, the unrotated inner product:

$$\left(\mathbf{W}_Q\,\text{Norm}(\mathbf{h})\right) \cdot \mathbf{K}_{\text{desc}}^{\top}$$

fails to correspond to the true post-RoPE attention scores, degrading retrieval accuracy.

To resolve this mismatch, PAKP explicitly applies RoPE to the lookahead Query using the current decoding token position p_{curr}:

$$\hat{\mathbf{Q}}_{L+1}^{\text{RoPE}} = \text{RoPE}\left(\hat{\mathbf{Q}}_{L+1},\, p_{\text{curr}}\right). \tag{52}$$

The rotation is computed using the model's native rotary embedding module (`model.model.rotary_emb`), ensuring numerical equivalence with the model's standard forward pass.

D. Grouped-Query Attention Structuring

Following RoPE transformation, the lookahead Query $\hat{\mathbf{Q}}_{L+1}^{\text{RoPE}} \in \mathbb{R}^{1\times H_Q\times d_h}$ is restructured to align with the Grouped-Query Attention topology:

$$\hat{\mathbf{Q}}_{\text{grouped}} \in \mathbb{R}^{H_{\text{kv}}\times G\times d_h}, \tag{53}$$

where $G = H_Q/H_{\text{kv}}$. Each of the H_{kv} Key-Value heads is evaluated against all G associated Query heads, allowing the bounding-box scoring function in Equation (36) to identify whether *any* query head within the group exhibits high affinity for a historical page.

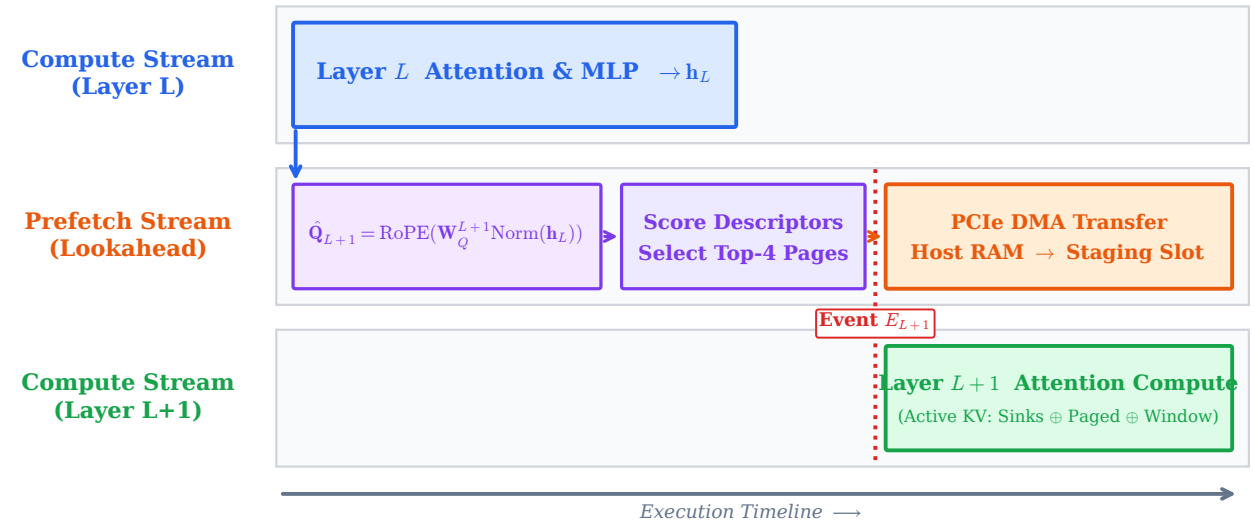


Fig. 5: Cross-layer lookahead pipeline showing execution swimlanes across Layer L and Layer $L+1$. While Layer L forward attention computes on $\mathcal{S}_{\text{compute}}$, the lookahead router projects $\hat{\mathbf{Q}}_{L+1}$ with RoPE alignment, evaluates bounding-box page scores, and dispatches asynchronous DMA prefetching on $\mathcal{S}_{\text{prefetch}}$, enabling zero-stall execution at Layer $L+1$.

E. Distinction: Hidden-State versus Query Continuity

We highlight a crucial theoretical distinction: *high cosine similarity in the residual stream does not imply equal cosine similarity in Query space*.

While $\mathbf{h}_L$ and $\mathbf{h}_{L+1}$ may exhibit cosine similarity exceeding 0.98 due to the additive nature of the residual connection, the Query projections at adjacent layers use independently learned weight matrices $\mathbf{W}_Q^{(L)}$ and $\mathbf{W}_Q^{(L+1)}$. Query vectors between Layer L and Layer $L+1$ are not expected to be colinear:

$$\mathrm{CosSim}\left(\mathbf{Q}_L, \mathbf{Q}_{L+1}\right) \ll \mathrm{CosSim}\left(\mathbf{h}_L, \mathbf{h}_{L+1}\right). \tag{54}$$

PAKP does *not* assume that $\mathbf{Q}_L \approx \mathbf{Q}_{L+1}$. Instead, PAKP projects $\mathbf{h}_L$ through the *actual* next-layer projection matrix $\mathbf{W}_Q^{(L+1)}$. The approximation error is strictly governed by the residual update magnitude $\|\Delta \mathbf{h}_L\|$, mapped through Layer $L+1$'s own Query projection:

$$\mathbf{Q}_{L+1} - \hat{\mathbf{Q}}_{L+1} = \mathbf{W}_Q^{(L+1)} \Delta\,\mathrm{Norm}_{L+1}, \tag{55}$$

where $\Delta\,\mathrm{Norm}_{L+1} = \mathrm{Norm}_{L+1}(\mathbf{h}_{L+1}) - \mathrm{Norm}_{L+1}(\mathbf{h}_L)$. This formulation preserves layer-specific Query semantic orientations while exploiting residual stream stability.

VII. Implementation and Integration

A. Hugging Face DynamicCache Subclassing

PAKP is implemented as a drop-in replacement for the standard Hugging Face Transformers KV cache by subclassing `DynamicCache` [10] into a custom class: `PAKPAsymmetricCache`.

The class overrides two primary methods and introduces two lifecycle hooks:

- **`update(key_states, value_states, layer_idx, ...)`:** During prefill (`is_decoding=False`), new token KV representations are accumulated contiguously in VRAM across all layers. During autoregressive decoding (`is_decoding=True`), single-token KV states are appended to the local VRAM window. The method synchronizes with the prefetch stream via a CUDA event wait on `prefetch_events` and returns the assembled active KV tensor $[\mathbf{K}_{\text{sink}} \oplus \mathbf{K}_{\text{prefetch}} \oplus \mathbf{K}_{\text{local}}]$.
- **`get_seq_length(layer_idx)`:** Returns the effective sequence length of the currently assembled active context:

$$S_{\text{active}} = |\mathbf{K}_{\text{sink}}| + |\mathbf{K}_{\text{prefetch}}| + |\mathbf{K}_{\text{local}}|. \tag{56}$$

- **`evict_prefill_to_pinned_host()`:** Executes post-prefill eviction, computes centered bounding-box descriptors, and migrates raw KV pages to pinned host RAM (detailed in Algorithm 1).
- **`trigger_lookahead_prefetch(layer_idx, q_rope_grouped)`:** Computes bounding-box and mean scores against the VRAM index and dispatches non-blocking DMA transfers on the prefetch stream (detailed in Algorithm 2).

Algorithm 1: Historical KV Eviction and Descriptor Construction

Input: Full prefill Key and Value caches $\mathbf{K}^{(l)}, \mathbf{V}^{(l)}$ for $l \in \{0, \dots, L-1\}$; parameters $W_{\text{sink}}, W_{\text{local}}, B$.
Output: Updated VRAM structures and pinned host page pool $\mathcal{H}^{(l)}$.

for $l \leftarrow 0$ **to** $L-1$ **do**
  $S_{\text{total}} \leftarrow \text{size}(\mathbf{K}^{(l)}, 2)$;
  $\mathbf{K}^{(l)}_{\text{sink}}, \mathbf{V}^{(l)}_{\text{sink}} \leftarrow \mathbf{K}^{(l)}[:,:,:W_{\text{sink}}], \mathbf{V}^{(l)}[:,:,:W_{\text{sink}}]$;
  $N_{\text{evict}} \leftarrow S_{\text{total}} - W_{\text{sink}} - W_{\text{local}}$;
  $P \leftarrow \max(0, \lfloor N_{\text{evict}}/B \rfloor)$;
  $E_{\text{end}} \leftarrow W_{\text{sink}} + P \cdot B$;
  $\mathbf{K}_{\text{mid}} \leftarrow \mathbf{K}^{(l)}[:,:,W_{\text{sink}}:E_{\text{end}}]$;
  $\mathbf{V}_{\text{mid}} \leftarrow \mathbf{V}^{(l)}[:,:,W_{\text{sink}}:E_{\text{end}}]$;
  $\mathbf{K}^{(l)}_{\text{local}}, \mathbf{V}^{(l)}_{\text{local}} \leftarrow \mathbf{K}^{(l)}[:,:,E_{\text{end}}:], \mathbf{V}^{(l)}[:,:,E_{\text{end}}:]$;
  `// Compute Centered Descriptors`
  $\bar{\mathbf{K}}^{(l)}_{\text{global}} \leftarrow \text{mean}(\mathbf{K}_{\text{mid}}, \text{dim} = \text{tokens})$;
  $\widetilde{\mathbf{K}}_{\text{mid}} \leftarrow \mathbf{K}_{\text{mid}} - \bar{\mathbf{K}}^{(l)}_{\text{global}}$;
  Reshape $\widetilde{\mathbf{K}}_{\text{mid}} \rightarrow [P, H_{\text{kv}}, B, d_h]$;
  $\boldsymbol{\mu}^{(l)}_p \leftarrow \text{mean}(\widetilde{\mathbf{K}}_{\text{mid}}, \text{dim} = B)$;
  $\mathbf{K}^{(p,l)}_{\text{min}} \leftarrow \min(\widetilde{\mathbf{K}}_{\text{mid}}, \text{dim} = B)$;
  $\mathbf{K}^{(p,l)}_{\text{max}} \leftarrow \max(\widetilde{\mathbf{K}}_{\text{mid}}, \text{dim} = B)$;
  `// Migrate Raw Pages to Pinned Host Memory`
  for $p \leftarrow 0$ **to** $P-1$ **do**
    Allocate pinned host buffer $\mathcal{H}^{(l)}_p \in \mathbb{R}^{2\times 1\times H_{\text{kv}}\times B\times d_h}$;
    $\mathcal{H}^{(l)}_p[0] \leftarrow \mathbf{K}_{\text{mid}}[:,:,p\cdot B:(p+1)\cdot B]$;
    $\mathcal{H}^{(l)}_p[1] \leftarrow \mathbf{V}_{\text{mid}}[:,:,p\cdot B:(p+1)\cdot B]$;
  end
  del $\mathbf{K}_{\text{mid}}, \mathbf{V}_{\text{mid}}$;
end
cudaDeviceSynchronize(); cudaEmptyCache();

Algorithm 2: Lookahead Prefetching and Dispatch

Input: Target layer t; lookahead grouped query $\hat{\mathbf{Q}}_{\text{grouped}} \in \mathbb{R}^{H_{\text{kv}}\times G\times d_h}$; prefetch budget K.
Output: Prefetched tensors $\mathbf{K}^{(t)}_{\text{prefetch}}, \mathbf{V}^{(t)}_{\text{prefetch}}$ in VRAM staging.

$P \leftarrow \text{len}(\mathcal{H}^{(t)})$;
if $P == 0$ **then**
  return;
end
$K_{\text{fetch}} \leftarrow \min(K, P)$;
`// Step 1: Compute Bounding-Box Upper Bound`
$\mathbf{P}_{\max}(p,h,g) \leftarrow \sum_d \hat{Q}_{h,g,d} K^{(p,t)}_{\max,h,d}$;
$\mathbf{P}_{\min}(p,h,g) \leftarrow \sum_d \hat{Q}_{h,g,d} K^{(p,t)}_{\min,h,d}$;
$S_{\text{bbox}}(p) \leftarrow \max_h \max_g \sum_d \max(\mathbf{P}_{\max}, \mathbf{P}_{\min})$;
`// Step 2: Compute Centered Mean Topic Score`
$S_{\text{mean}}(p) \leftarrow \frac{1}{H_{\text{kv}}G} \sum_{h,g,d} \hat{Q}_{h,g,d} \mu^{(t)}_{p,h,d}$;
$S_{\text{PAKP}}(p) \leftarrow S_{\text{bbox}}(p) + \lambda \cdot S_{\text{mean}}(p)$;
$\mathcal{I}_{\text{top}} \leftarrow \text{Top-K}(\{S_{\text{PAKP}}(p)\}_{p=0}^{P-1}, K_{\text{fetch}})$;
`// Step 3: Asynchronous Host-to-Device DMA`
Enqueue non-blocking DMA transfers on $\mathcal{S}_{\text{prefetch}}$;
$\mathcal{K}_{\text{list}}, \mathcal{V}_{\text{list}} \leftarrow [], []$;
for $idx \in \mathcal{I}_{\text{top}}$ **do**
  $\mathcal{K}_{\text{list}} \leftarrow \mathcal{K}_{\text{list}} \cup \{\mathcal{H}^{(t)}_{idx}[0] \xrightarrow{\text{DMA}} \text{VRAM}\}$;
  $\mathcal{V}_{\text{list}} \leftarrow \mathcal{V}_{\text{list}} \cup \{\mathcal{H}^{(t)}_{idx}[1] \xrightarrow{\text{DMA}} \text{VRAM}\}$;
end
$\mathbf{K}^{(t)}_{\text{prefetch}} \leftarrow \text{concat}(\mathcal{K}_{\text{list}}, \text{dim} = \text{tokens})$;
$\mathbf{V}^{(t)}_{\text{prefetch}} \leftarrow \text{concat}(\mathcal{V}_{\text{list}}, \text{dim} = \text{tokens})$;
E_t.record($\mathcal{S}_{\text{prefetch}}$);

B. Computational and Space Complexity Analysis

We analyze the computational complexity and memory consumption of the PAKP components:

1) Descriptor Construction (One-time post-prefill): For each of the P historical pages across L layers:

- Global mean computation: $\mathcal{O}(L \cdot P \cdot B \cdot H_{\text{kv}} \cdot d_h)$ FLOPs.
- Centering subtraction: $\mathcal{O}(L \cdot P \cdot B \cdot H_{\text{kv}} \cdot d_h)$ FLOPs.
- Per-page reduction (mean, amin, amax): $3 \times \mathcal{O}(L \cdot P \cdot B \cdot H_{\text{kv}} \cdot d_h)$ operations.

Because this routine executes exactly once after prompt prefill, its amortized cost over generation is negligible (< 15 ms total execution time on the RTX 4050 for 2,655 tokens).

2) Page Scoring and Top-K Selection (Per decoding token): At each layer l, matching the lookahead Query $\hat{\mathbf{Q}}_{L+1}$ against the GPU-resident index requires:

- Bounding-box projections: Two batched matrix multiplications of sizes $(H_{\text{kv}} \cdot G, d_h)$ and $(P \cdot H_{\text{kv}}, d_h)$, requiring $2 \cdot P \cdot H_Q \cdot d_h$ FLOPs.
- Element-wise maximum, summation, and head reduction: $\mathcal{O}(P \cdot H_Q \cdot d_h)$ operations.
- Centered mean projection: $P \cdot H_Q \cdot d_h$ FLOPs.
- Top-K selection: $\mathcal{O}(P + K \log K)$ operations via max-heap selection.

For $P = 512$ pages, $H_Q = 12$, and $d_h = 128$, the total scoring cost per layer is approximately:

$$\approx 4 \times 512 \times 12 \times 128 \approx 3.15 \times 10^6 \text{ FLOPs} \approx 3.15 \text{ MFLOPs}. \tag{57}$$

On an RTX 4050 GPU with tens of TFLOPs tensor compute, evaluating this search takes under 0.05 ms per layer, adding minimal overhead to the critical path.

3) Attention Computation Over Active Tokens: Standard attention over the uncompressed sequence of S tokens requires $\mathcal{O}(S \cdot H_Q \cdot d_h)$ operations per token. Under PAKP, attention is computed only over $S_{\text{active}} = W_{\text{sink}} + K \cdot B + W_{\text{local}}$ tokens:

$$\mathcal{C}_{\text{attn}} = \mathcal{O}\left((W_{\text{sink}} + K \cdot B + W_{\text{local}}) \cdot H_Q \cdot d_h\right). \tag{58}$$

For $S = 32{,}768$ versus $S_{\text{active}} = 516$, the attention computation cost per generated token is reduced by:

$$\frac{32{,}768}{516} \approx 63.5\times . \tag{59}$$

4) Space Complexity: The total GPU memory required by PAKP during decoding scales with the number of pages P as:

$$M_{\text{GPU}}^{\text{PAKP}}(P) = \mathcal{O}\big(M_{\text{weights}} + S_{\text{active}} L H_{\text{kv}} d_h + P \cdot L \cdot H_{\text{kv}} \cdot d_h\big). \tag{60}$$

Because the page descriptor size per page ($3 \times H_{\text{kv}} \times d_h \times 2$ bytes $= 6{,}144$ bytes/page) is $21.3\times$ smaller than the raw KV cache of that page ($2 \times 64 \times H_{\text{kv}} \times d_h \times 2$ bytes $= 131{,}072$ bytes/page), the rate of GPU memory growth with context length is reduced by over 95%.

VIII. Experimental Evaluation

The empirical evaluation of PAKP is organized into three distinct categories reflecting the research progression:

1) **Category A: Synthetic Experiments** (§VIII-B1 and §VIII-B2), evaluating transfer-latency hiding and lookahead routing properties under controlled synthetic tensor distributions.
2) **Category B: Live Pretrained Model Experiments** (§VIII-C), evaluating end-to-end integration with `Qwen2.5-1.5B-Instruct` on an NVIDIA RTX 4050 GPU, including diagnosis of an initial failure mode and a successful passkey retrieval test.
3) **Category C: Multi-Method Ablation Study** (§VIII-D), reporting preliminary comparative ablation results and documenting an out-of-memory bottleneck during prefill.

A. Experimental Environment

All experiments were conducted on a commodity laptop workstation with the hardware and software configuration specified in Table II.

TABLE II: Experimental Hardware and Software Environment

Component	Specification / Version
Host Processor	Intel Core i7-13620H (10 cores, 16 threads)
System Memory (RAM)	32.0 GB DDR5
Dedicated Accelerator	NVIDIA GeForce RTX 4050 Laptop GPU
GPU VRAM	6.00 GB GDDR6 (5.71 GiB physical limit)
Interconnect	PCIe Gen4 ×8
Measured H2D Bandwidth	11.81 GB/s (Pinned RAM → VRAM)
Operating System	Linux (Kernel 6.x)
Deep Learning Framework	PyTorch 2.x, CUDA 12.x
Model Library	Hugging Face Transformers 4.4x
Target LLM	Qwen2.5-1.5B-Instruct
Precision	FP16 (torch.float16)

B. Category A: Synthetic Experiments

1) Synthetic Dual-Stream Memory Benchmark: The first synthetic experiment, implemented in `pakp_memory_pool.py`, evaluates whether dual CUDA streams can hide PCIe transfer latency behind GPU matrix multiplication and attention computation.

Experimental Setup. The benchmark simulates a 28-layer Transformer ($L = 28, H_{\text{kv}} = 8, d_h = 128$) with a 32,768-token context window ($P = 512$ pages, $B = 64$). A pinned host memory pool of 3.50 GB is pre-allocated. Per-layer computation simulates single-token Grouped-Query Attention (24 query heads, 8 KV heads) concatenated with SwiGLU MLP projection ($3072 \to 8192 \to 3072$).

Four execution modes were evaluated:

1) **Naive Full KV Offload (Calculated Baseline):** Computed as the sum of measured GPU forward computation plus the theoretical host-to-device PCIe transfer time of the complete 3.50 GB KV cache (296.36 ms at 11.81 GB/s).
2) **Ideal In-VRAM Compute (Upper Bound):** Measures purely GPU forward computation assuming all 772 active tokens reside in VRAM with zero PCIe transfer.
3) **Synchronous Top-4 Paging:** For each layer, synchronously transfers 4 pages (256 tokens) from pinned host RAM to VRAM on the default stream before executing layer compute.
4) **PAKP Lookahead Dual-Stream:** Prefetches Layer $L+1$'s 4 pages asynchronously on $\mathcal{S}_{\text{prefetch}}$ while computing Layer L on $\mathcal{S}_{\text{compute}}$, synchronizing via CUDA events.

Results. Table III and Figure 6 report the latency and throughput results across 28 layers.

TABLE III: Synthetic Dual-Stream Benchmark Results (32,768 Context)

Execution Mode	Latency (ms/tok)	Throughput (tok/s)
1. Naive Full Offload†	321.26	3.1
2. Ideal In-VRAM Compute	24.90	40.2
3. Synchronous Top-4 Paging	31.60	31.6
4. PAKP Dual-Stream (Ours)	**25.54**	**39.2**

†Derived from theoretical PCIe transfer estimate (296.36 ms) plus compute.

The latency overhead imposed by synchronous paging over ideal in-VRAM compute is:

$$T_{\text{sync_overhead}} = 31.60 - 24.90 = 6.70 \text{ ms/token}. \tag{61}$$

Under PAKP dual-stream lookahead prefetching, the observed per-token latency drops to 25.54 ms, reducing the transfer overhead to:

$$T_{\text{pakp_overhead}} = 25.54 - 24.90 = 0.64 \text{ ms/token}. \tag{62}$$

The percentage of PCIe transfer latency hidden behind computation is:

$$\eta_{\text{hidden}} = \frac{31.60 - 25.54}{31.60 - 24.90} \times 100\% = \frac{6.06}{6.70} \times 100\% \approx 90.45\%. \tag{63}$$

Active VRAM allocated during this benchmark was measured at 182.45 MB (including synthetic MLP parameters and buffers).

Critical Qualification. We emphasize that Mode 1 represents a calculated theoretical estimate rather than a directly timed full-offload baseline. Furthermore, this benchmark uses

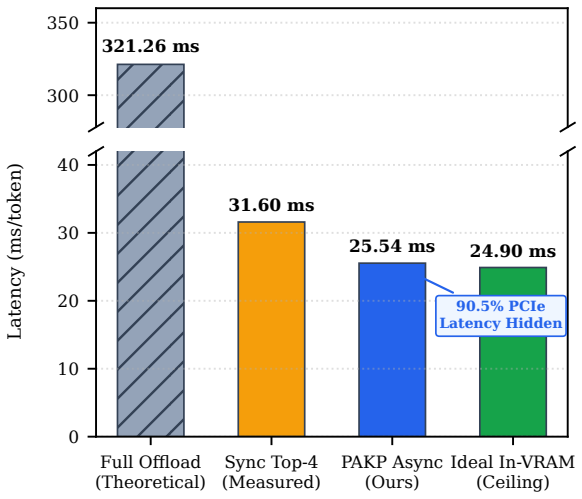


Fig. 6: Synthetic memory-pool benchmark comparing per-token decoding latency across 28 layers with a 32,768-token context. PAKP dual-stream prefetching achieves 25.54 ms/token, approaching the in-VRAM theoretical limit (24.90 ms/token).

randomly initialized tensors and hardcoded page indices; it evaluates DMA-compute overlap mechanics, not semantic retrieval accuracy.

2) Synthetic Lookahead Routing Evaluation: The second synthetic experiment, implemented in `pakp_lookahead_eval.py`, evaluates cross-layer residual stream continuity and page-retrieval recall under controlled needle-implant conditions.

Experimental Setup. The simulation models 28 layers with $d_{\text{model}} = 3072$, $H_{\text{kv}} = 8$, and $d_h = 128$. Per-layer Query projections $\mathbf{W}_Q^{(l)}$ are randomly initialized. The residual stream trajectory is simulated with a controlled per-layer drift parameter $\delta = 0.18$, modeling typical relative residual updates $\|\Delta \mathbf{h}_L\|/\|\mathbf{h}_L\|$ observed in Llama/Qwen architectures:

$$\mathbf{h}_{L+1} = \text{Norm}\left(\mathbf{h}_L + \delta \cdot \text{Norm}(\epsilon_L)\right) \cdot \sqrt{d_{\text{model}}}, \tag{64}$$

where $\epsilon_L \sim \mathcal{N}(0, \mathbf{I})$.

Three semantic needle pages ($p \in \{17, 204, 419\}$) were implanted among 512 total pages. Within each needle page, a single token at position 31 was injected with strong directional alignment to the query trajectory. Page selection was performed using the lookahead Query $\hat{\mathbf{Q}}_{L+1}$ projected from $\mathbf{h}_L$ through $\mathbf{W}_Q^{(L+1)}$.

Results. Table IV reports the measured telemetry across all 27 layer transitions.

TABLE IV: Synthetic Lookahead Routing Telemetry

Evaluation Metric	Measured Result
Average Residual Stream Cosine Similarity ($\mathbf{h}_L$ vs $\mathbf{h}_{L+1}$)	0.9842
Naive Mean-Centroid Lookahead Needle Recall	100.00%
PAKP Dual-Descriptor (Mean + Max) Needle Recall	100.00%
Top-4 Page Agreement with Exact Oracle ($\mathbf{h}_{L+1}$)	75.00%

Analysis. The average cosine similarity of 0.9842 confirms strong directional continuity in the simulated residual stream. Both naive mean-centroid and dual-descriptor routing achieved 100% recall of the implanted needle pages across all evaluated layers.

Importantly, because both methods achieved 100% needle recall in this experiment, this result *does not* demonstrate an empirical retrieval advantage of the dual-descriptor representation over the mean centroid in this synthetic setup. The high recall is attributable to the strong artificial needle signal injected into the synthetic Key tensor.

The Top-4 oracle agreement metric was measured at 75.00%, indicating that on average, 3 out of the 4 pages selected by the predictive lookahead matched the set selected by the exact next-layer Query. Needle recall (finding specific target pages) and oracle agreement (reproducing the full Top-K ranking) measure distinct properties.

C. Category B: Live Pretrained Model Experiments

1) Initial Integration Failure and Diagnosis: The first end-to-end integration of PAKP with a pretrained language model (`Qwen2.5-1.5B-Instruct`) exposed a critical decoding degeneration. Although the custom cache successfully evicted 39 pages (2,496 tokens), triggered 448 lookahead prefetches, and reduced active KV tokens to 530 at a decoding speed of 27.6 tokens/sec (peak VRAM: 3.18 GB), the model generated repetitive degenerate text:

> `"The code is the the the the the the..."`

The passkey retrieval test failed completely.

Technical diagnosis identified two structural causes:

1) **Cache-Position and Mask Misalignment:** Hugging Face Transformers constructs causal attention masks and RoPE position sequences based on the value returned by `cache.get_seq_length()`. In the initial prototype, reporting a compressed cache length caused the model to apply causal masks that misaligned historical token coordinates with the current decoding position. Furthermore, an early chunked-prefill implementation performed eviction mid-prompt, depriving later prompt tokens of full causal context.
2) **Pre-RoPE versus Post-RoPE Representation Mismatch:** The initial lookahead router compared an unrotated Query projection against stored Key descriptors. However, in Qwen2.5, Keys stored during prefill have already undergone Rotary Position Embedding (RoPE). Comparing unrotated Query vectors against position-rotated Key centroids produced meaningless similarity scores.

These issues were resolved by: (i) preserving full-context prefill before eviction, (ii) explicitly applying the model's native RoPE module to the lookahead Query (Equation (52)), and (iii) introducing centered channel-wise bounding-box descriptors.

2) Successful Live Passkey Retrieval: Following these architectural revisions, PAKP was evaluated on a controlled long-context passkey retrieval task using `pakp_hf_live.py`.

Prompt Construction. A passkey phrase:

```
"The secret launch authorization
code for Project Titan is
7392-OMEGA."
```

was embedded inside a 2,655-token prompt composed of technical system logs. The secret passkey was located at token position 338.

Under the configured cache geometry ($W_{\text{sink}} = 4, W_{\text{local}} = 256, B = 64$), the target passkey falls within historical evicted page p_{target}:

$$p_{\text{target}} = \left\lfloor \frac{338 - 4}{64} \right\rfloor = \left\lfloor \frac{334}{64} \right\rfloor = 5. \tag{65}$$

Historical Page 5 was evicted to pinned host RAM after prefill. Correct passkey reproduction required the lookahead router to identify Page 5 and prefetch it into VRAM during decoding.

Measured Live Results. Table V and Figure 7 present the measured live results.

TABLE V: Live Model Passkey Retrieval (Qwen2.5-1.5B-Instruct, FP16)

Metric	Observed Measurement
Total Prompt Length	2,655 tokens
Ground-Truth Target Token	Position #338
Target Historical Page Index	Page #5 (Evicted to Host RAM)
Number of Evicted Host Pages	37 pages (2,368 tokens in RAM)
Router Selected Pages (Layer 16)	**[0, 1, 2, 5]**
Target Page Hit in Top-4	**Yes** ($5 \in \{0, 1, 2, 5\}$)
Active KV Tokens in VRAM	**552 tokens** (vs. 2,655 prompt)
Active KV-Token Reduction	**79.21%**
Steady-State Decoding VRAM	3.64 GB (within 5.71 GiB budget)
Decoding Generation Speed	31.8 tokens/sec
Model Generated Output	`"7392-OMEGA."`
Exact Passkey Match	**Yes (100% exact match)**

The router at retrieval Layer 16 selected pages $[0, 1, 2, 5]$, successfully capturing the target Page 5. The model subsequently generated the exact target string:

```
"The secret launch authorization
code for Project Titan is
7392-OMEGA."
```

The active KV-token footprint in VRAM decreased from 2,655 to 552 tokens, representing an active KV-token reduction of:

$$R_{\text{KV}} = \left(1 - \frac{552}{2{,}655}\right) \times 100\% \approx 79.21\%. \tag{66}$$

Critical Qualification. This result constitutes a successful feasibility demonstration of end-to-end lookahead-driven historical page retrieval on a live model. It demonstrates that the centered bounding-box descriptor and RoPE-aligned lookahead router can successfully identify and reload an evicted historical page. However, this is a single controlled test case; it does *not* establish universal passkey retrieval accuracy across diverse context lengths, positions, or prompts.

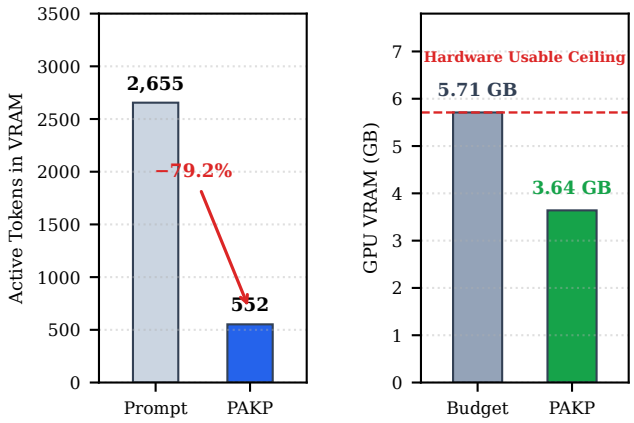


Fig. 7: Live model evaluation on Qwen2.5-1.5B-Instruct. (Left) Active KV-cache token count in VRAM is reduced from 2,655 to 552 tokens (79.2% reduction). (Right) Steady-state VRAM allocation of 3.64 GB remains well within the 5.71 GiB usable capacity of the RTX 4050 GPU.

D. Category C: Multi-Method Ablation Study

To isolate the contribution of the centered bounding-box descriptor, a multi-method ablation benchmark was developed in `pakp_ablation_suite.py`, comparing:

1) **Sliding Window Baseline:** Retains attention sinks ($W_{\text{sink}} = 4$) and local window ($W_{\text{local}} = 256$); no historical pages are prefetched.
2) **Naive Mean-Centroid Paging:** Uses uncentered mean descriptors (Equation (29)) for lookahead page selection.
3) **PAKP Bounding-Box Paging:** Uses centered bounding-box plus mean descriptors (Equation (39)).

Execution and Observed Failure. The ablation suite was executed on a 3,567-token prompt with the passkey located in Page 6. Table VI reports the observed output.

Diagnosis of the Out-Of-Memory Failure. The sliding-window baseline completed successfully, generating `"123456"` without retrieving the passkey (as expected, since Page 6 was outside the active window).

However, during execution of the second configuration (Naive Mean-Centroid), the process terminated with a CUDA Out-Of-Memory exception:

```
torch.OutOfMemoryError: CUDA out of
memory. Tried to allocate 1.01 GiB
(GPU 0; 5.71 GiB total capacity;
2.99 GiB already allocated; 862.00 MiB
free; 1.01 GiB reserved by PyTorch).
```

The failing operation in the PyTorch stack trace was:

```
logits = self.lm_head(hidden_states)
```

The cause of this failure is the materialization of full-sequence vocabulary logits during prefill. In `pakp_ablation_suite.py`, the prefill step invokes the full causal language model wrapper:

$$\mathbf{L}_{\text{vocab}} = \mathbf{H}\mathbf{W}_{\text{LM}}^{\top} \quad \in \mathbb{R}^{B_{\text{batch}} \times S_{\text{prompt}} \times V}, \tag{67}$$

TABLE VI: Multi-Method Ablation Benchmark (Prompt Context: 3,567 tokens; Target Page: #6)

Configuration	Selected Pages	Page Hit	Peak VRAM	Throughput	Generated Output	Status
1. Sliding Window	`[]`	False	3.90 GB	46.6 tok/s	`"123456"`	Completed (Failed retrieval)
2. Naive Mean-Centroid	—	—	OOM (>5.71 GB)	—	—	**Failed: CUDA OOM during prefill**
3. PAKP Bounding-Box	—	—	—	—	—	**Not reached in this run**

where $S_{\text{prompt}} = 3{,}566$ tokens, vocabulary size $V \approx 152{,}064$ (for Qwen2.5), and elements are in FP16 (2 bytes). The memory required for this single temporary tensor is:

$$M_{\text{logits}} = 1 \times 3{,}566 \times 152{,}064 \times 2 \text{ bytes} \approx 1.085 \times 10^9 \text{ bytes} \approx 1.01 \text{ GiB}. \quad (68)$$

Because the model weights (≈ 3.0 GB) and KV cache were already resident, allocating an additional 1.01 GiB for vocabulary logits exceeded the 862 MiB of free GPU memory.

Importantly, language model prefill requires only updating the KV cache; materializing vocabulary logits for prompt tokens is unnecessary during inference. Bypassing the language model head by calling the decoder backbone directly (`model.model(input_ids, ...)`) would eliminate this 1.01 GiB allocation.

Because the benchmark terminated at this point, the naive mean-centroid configuration was not evaluated for retrieval quality, and the PAKP bounding-box configuration was not reached. We report this outcome factually: *the comparative ablation remains incomplete, and no empirical claim of superiority over mean-centroid paging can be established from this benchmark run.*

IX. Discussion

A. Synthesis of Experimental Findings

The empirical evaluation provides a clear picture of what the PAKP prototype accomplishes and what remains to be verified:

1) **Transfer Latency Hiding is Mechanistically Feasible:** The synthetic benchmark (§VIII-B1) demonstrated that an asynchronous prefetch stream can hide approximately 90.5% of host-to-device PCIe transfer latency behind GPU computation during autoregressive decoding. The dual-stream pipeline achieved 25.54 ms/token, closely approaching the ideal in-VRAM ceiling of 24.90 ms/token.
2) **Lookahead Query Prediction is Directionally Sound:** The synthetic routing evaluation (§VIII-B2) confirmed an average residual-stream cosine similarity of 0.9842 across adjacent layers. Projecting the preceding layer's hidden state through the subsequent layer's Query projection produced sufficient directional alignment to achieve 75% Top-4 page agreement with the exact oracle.
3) **End-to-End Live Feasibility Demonstrated:** The live experiment on `Qwen2.5-1.5B-Instruct` (§VIII-C2) validated the complete system stack under real model weights and activations. The lookahead router successfully identified the evicted historical page containing a passkey, transferred it to GPU memory, and enabled the model to reproduce the exact passkey. Active KV tokens in VRAM were reduced by 79.21%, and steady-state VRAM consumption remained at 3.64 GB, well within the 6 GB GPU budget.
4) **The Necessity of Structural Alignments:** The initial live failure (§VIII-C1) provided a key architectural lesson: custom KV-cache management cannot treat Query and Key representations as abstract vector spaces. Rotary Position Embeddings rotate representations in position-dependent subspaces; failing to apply RoPE to the predictive Query corrupts page scoring. Similarly, custom caches must carefully manage cache length reporting to maintain causal mask alignment.

B. Comparison with Related Paradigms

Table VII compares PAKP with existing KV-cache management paradigms across key architectural dimensions.

1) PAKP versus Token Eviction (StreamingLLM, H_2O): Token eviction methods permanently delete historical tokens to bound memory. While effective for continuous streaming or summarization of recent context, discarded information is irrecoverable. If a user query requires recalling a specific detail presented early in a document, eviction architectures fail unconditionally. PAKP preserves the entire history in host RAM, enabling selective retrieval when needed.

2) PAKP versus Query-Aware Selection (Quest): Quest [5] pioneered tracking min/max Key bounds for query-aware page selection. However, Quest is designed for datacenter GPUs where the entire KV cache resides in GPU High-Bandwidth Memory (HBM); its objective is reducing memory bandwidth during the attention kernel, not offloading across the PCIe bus. Quest performs page selection synchronously at each layer using the true Query.

In contrast, PAKP targets consumer GPUs where the KV cache cannot fit in VRAM. This setting introduces two distinct architectural requirements: (i) pages must be transferred across the PCIe bus, requiring *asynchronous predictive prefetching* to hide transfer latency, and (ii) Key representations are centered globally ($\bar{\mathbf{K}}_{\text{global}}$) to eliminate anisotropic coordinate bias before bounding-box evaluation.

3) PAKP versus Full Offloading (FlexGen): FlexGen [6] moves weights and KV caches between GPU, CPU, and disk, optimizing for throughput over large batches. However, for interactive single-stream generation, transferring full KV caches across PCIe introduces hundreds of milliseconds of latency per token. PAKP reduces transferred data by two orders of magnitude ($128\times$ in our 32K configuration) by transferring only predicted critical pages.

TABLE VII: Architectural Comparison of KV-Cache Management Paradigms

Paradigm / System	Storage Tier	Eviction Strategy	History Accessible?	Hardware Target	Prefetching?	Search Mechanism
vLLM / PagedAttention [7]	GPU HBM	Dynamic block paging	Yes (in VRAM)	Datacenter GPUs	No	Page table lookup
FlexGen [6]	GPU/CPU/Disk	Linear program offload	Yes (batch-delayed)	Single GPU	Bulk overlap	None (full transfer)
StreamingLLM [2]	GPU VRAM	Sinks + local window	**No (permanently dropped)**	Any	No	None (static window)
H_2O [3]	GPU VRAM	Heavy-hitter score	**No (permanently dropped)**	Any	No	Accumulated attention
SnapKV [4]	GPU VRAM	Observation clustering	**No (compressed in-place)**	Any	No	Prompt-end voting
Quest [5]	GPU HBM	Min/max page bounds	Yes (in HBM)	Server GPUs	No (synchronous)	Channel bounding box
PAKP (Ours)	**GPU + Host RAM**	**Centered Bounding-Box**	**Yes (retained in Host)**	**Consumer GPUs**	**Cross-layer lookahead**	**RoPE-aligned Lookahead**

C. System Implications for Consumer Hardware

The broader significance of PAKP lies in consumer-grade accessibility. High-end datacenter GPUs (such as NVIDIA H100 with 80 GB HBM3) can easily accommodate 32K or 64K KV caches directly in VRAM. However, consumer hardware (6 GB to 12 GB VRAM) remains severely restricted.

By pairing 6 GB of fast GPU VRAM with 32 GB of inexpensive DDR5 system RAM, PAKP enables long-context inference on commodity laptop and desktop workstations. The steady-state memory footprint observed in our live experiments (3.64 GB) indicates that 1.5B to 3B models can operate comfortably within a 6 GB VRAM envelope while retaining access to tens of thousands of historical tokens stored in host RAM.

X. Limitations and Threats to Validity

In accordance with academic rigor, we explicitly articulate the limitations of the current prototype and the boundaries of the empirical evidence.

A. Incomplete Multi-Method Ablation

As documented in Section VIII-D, the multi-method ablation suite terminated prematurely during the prefill stage of the naive mean-centroid configuration due to a CUDA Out-Of-Memory error caused by full-sequence vocabulary logit materialization. Consequently:

- The naive mean-centroid method was not evaluated on retrieval accuracy or decoding speed in the ablation suite.
- The PAKP bounding-box configuration was not reached in that benchmark execution.

Therefore, this work *does not present empirical ablation proof* that the centered bounding-box descriptor outperforms naive mean-centroid paging on the live model across diverse context lengths. While the bounding-box descriptor demonstrated successful passkey retrieval in the isolated live test (§VIII-C2) and 100% recall in synthetic testing (§VIII-B2), a rigorous head-to-head comparison against mean-centroid paging under identical prefill conditions remains to be completed.

B. Scope of Live Pretrained Model Evaluation

The live model evaluation in Section VIII-C2 is subject to several empirical constraints:

1) **Single Model Architecture:** Live testing was conducted exclusively on `Qwen2.5-1.5B-Instruct`. Validation on larger models (such as Llama-3.2-3B or Qwen2.5-7B) has not been performed due to VRAM constraints.
2) **Single Passkey Instance:** Testing evaluated a single passkey string (`"7392-OMEGA"`) placed at a single historical position (token #338, Page #5) within a 2,655-token context. Testing across varied needle positions (e.g., 10%, 25%, 50%, 75%, 90% context depth) and multiple random seeds is required to establish general retrieval robustness.
3) **Absence of Distractors:** The evaluation prompt contained a single semantic target. In scenarios with multiple competing semantic targets or distracting technical logs, page scoring may exhibit false positives.
4) **No Language Modeling Perplexity Benchmark:** We did not measure perplexity degradation on standard language modeling benchmarks (e.g., WikiText-103, PG-19) under PAKP paging.

C. Implementation and System Constraints

1) **Prefill Logit Materialization:** Invoking `model(input_ids, ...)` during prefill unnecessarily materializes logits of size $[1, S_{\text{prompt}}, 152064]$, consuming over 1 GiB of temporary VRAM at $S = 3{,}566$. Bypassing the language model head via `model.model` is necessary to scale prefill beyond 3.5K tokens on a 6 GB GPU.
2) **Unbounded Local Window Growth During Decoding:** In the current implementation, newly generated tokens are appended to `vram_window_k` without an eviction trigger during decoding. For short generation sequences ($\leq$ 12 tokens), this footprint is negligible; however, long autoregressive generations would eventually exhaust VRAM unless an incremental circular eviction policy is implemented.
3) **Non-Contiguous Position Handling in Attention:** Assembled active KV tensors contain tokens from non-contiguous sequence positions (sinks at 0–3, historical pages at arbitrary positions, and local window at recent positions). While each token retains its individual RoPE-rotated Key representation from prefill, standard FlashAttention or PyTorch SDPA kernels treat the concatenated sequence as contiguous. The mathematical impact of non-contiguous relative distance steps across block boundaries requires further theoretical characterization.
4) **Transfer Overlap Profiling:** While transfer-compute overlap was directly verified in the synthetic benchmark (§VIII-B1), hardware-level profiling via NVIDIA Nsight Systems has not yet been collected for the live Hugging

Face implementation to verify that PCIe DMA transfers do not experience kernel launch serialization or PyTorch GIL contention.

D. Future Research Directions

Addressing these limitations defines the roadmap for future research:

- **Bypassing the LM Head During Prefill:** Refactoring the prefill pipeline to call the decoder backbone directly, enabling full-context prefill up to 16K–32K tokens within the 6 GB budget.
- **Comprehensive Multi-Scale Ablation:** Completing comparative benchmarks across 2K, 4K, 8K, 16K, and 32K context lengths across Sliding Window, Mean-Centroid, and PAKP Bounding-Box configurations.
- **Standardized Long-Context Benchmarks:** Evaluating retrieval fidelity on the Needle-In-A-Haystack (NIAH) benchmark, LongBench, and Ruler suites.
- **Rolling Eviction During Generation:** Adding circular buffer eviction to the local window during autoregressive decoding.
- **Custom Fused Kernel:** Developing a Triton or CUDA kernel that fuses bounding-box scoring directly into the layer norm and projection kernels.

XI. Conclusion

This paper introduced Predictive Asymmetric KV-Paging (PAKP), a memory-management architecture designed to address the KV-cache memory bottleneck in long-context Transformer inference on consumer-grade GPUs. By pairing a capacity-constrained GPU with system RAM across an asymmetric hierarchy, PAKP maintains the complete historical KV cache in pinned host memory while keeping only a compact descriptor index, attention sinks, and a local context window in GPU VRAM.

The architecture integrates three core mechanisms:

1) A *centered bounding-box descriptor* that combines global Key centering with per-page channel-wise minimum and maximum extrema to provide an upper-bound relevance score, preserving isolated high-salience token signals that are diluted by mean pooling.
2) A *cross-layer predictive lookahead router* that projects the incoming hidden state through the subsequent layer's Query weights and applies RoPE alignment to predict historical page requirements before the target layer executes.
3) An *asynchronous dual-stream prefetching pipeline* that overlaps host-to-device PCIe page transfers with GPU forward computation via a dedicated CUDA stream.

Empirical evaluation established the following findings:

- Synthetic benchmarks demonstrated that asynchronous prefetching can hide approximately 90.5% of PCIe transfer latency behind GPU computation, reducing per-token decoding latency from 31.60 ms (synchronous) to 25.54 ms, approaching the in-VRAM ceiling of 24.90 ms.
- Synthetic lookahead routing confirmed an average adjacent-layer residual stream cosine similarity of 0.9842 and a 75.00% Top-4 page agreement with the exact oracle.
- Live evaluation on `Qwen2.5-1.5B-Instruct` running on an NVIDIA RTX 4050 Laptop GPU (6 GB VRAM) demonstrated successful end-to-end passkey retrieval from an evicted historical page in pinned host RAM. Active KV tokens in VRAM were reduced by 79.21% (from 2,655 to 552 tokens), and steady-state VRAM consumption remained bounded at 3.64 GB.
- Multi-method ablation testing documented an out-of-memory bottleneck arising from full-sequence vocabulary logit materialization during prefill, illustrating the critical importance of memory efficiency in the prefill pipeline on constrained devices.

These results provide an encouraging proof-of-concept for predictive asymmetric paging. While comprehensive multi-context ablation and broader benchmark evaluation remain necessary, PAKP demonstrates that intelligent lookahead routing and compact geometric descriptors can bridge the gap between consumer GPU memory limits and long-context Transformer inference.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.17453.

[3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2306.14048.

[4] Y. Li, Y. Huang, B. Yang, B. Vber, T. Hashimoto, R. Vuduc, and B. Chen, "SnapKV: LLM knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.

[5] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context LLM inference," in *International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.10774.

[6] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2303.06865.

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[9] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024, arXiv:2104.09864.

[10] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," pp. 38–45, 2020.